

Documento de Especificación del Diseño

Reventa B2B de tickets con NFTs en Stellar (Soroban)

(Proyecto Tickets-NFT B2B)

Versión 0.6 — 19 de mayo de 2026



Pontificia Universidad Javeriana Bogotá

Trabajo de Grado – Ingeniería de Sistemas

Director: Rafael Vicente Paéz Méndez, PhD

Cliente: B.A.F: Blockchain Acceleration Foundation

Autores:

Juan Diego Arias Durán

Santiago Andrés Mesa Niño

Andrés Felipe Manosalva Garzón

Juan Camilo Carvajal Rodríguez

Historial de cambios

Versión	Fecha	Descripción	Autor(es)
0.1	10/03/2026	Versión inicial del SAD: estructura base del documento.	Grupo 12
0.2	13/03/2026	Desarrollo de las secciones 1 (Introducción), 2 (Representación arquitectónica) y 3 (Objetivos y restricciones arquitectónicas).	Grupo 12
0.3	15/03/2026	Desarrollo de las secciones 4 (Vista de casos de uso), 5 (Vista lógica) y 6 (Vista de procesos).	Grupo 12
0.4	24/03/2026	Actualización integral: alineación con la implementación real de los contratos (nombres en español, versionado burn/remint, sistema de verificadores, invalidación on-chain), corrección de la arquitectura Frontend→Backend→Blockchain, eliminación de App de Escaneo y modo offline no implementados, actualización de diagramas C4.	Grupo 12
0.5	25/04/2026	Sincronización con estado final del prototipo: corrección de stack backend (Express en lugar de NestJS), documentación de integración Soroban completa (venta primaria, reventa atómica, asegurar on-chain, mercado secundario P2P, cancelación de listado), incorporación de Freightier Wallet como componente implementado, actualización de modelo de datos off-chain con campos Web3, adición de rol STAFF y panel de administración (Phase 4), corrección de flujo de verificación (escáner QR implementado, sin modo offline), actualización de rutas frontend y variables de entorno.	Grupo 12

Versión	Fecha	Descripción	Autor(es)
0.6	11/05/2026	Referencias bibliográficas movidas de la sección 1.4 al final del documento como sección independiente. Diagrama de casos de uso actualizado: solo Admin Plataforma puede crear eventos y desplegar contratos inteligentes. Actor renombrado de Comprador a Usuario en todos los casos de uso.	Grupo 12

Índice

1. Introducción	1
1.1. Propósito	1
1.2. Alcance	1
1.3. Definiciones, acrónimos y abreviaturas	1
1.4. Visión general del documento	2
2. Representación arquitectónica	3
2.1. Niveles C4 utilizados	3
2.1.1. Nivel 1 – Contexto del sistema	3
2.1.2. Nivel 2 – Contenedores	4
2.2. Mapeo de vistas a secciones del documento	6
3. Objetivos y restricciones arquitectónicas	8
3.1. Objetivos arquitectónicos	8
3.2. Restricciones arquitectónicas	9
3.2.1. Restricciones tecnológicas	9
3.2.2. Restricciones organizacionales	9
3.2.3. Restricciones de integración	9
4. Vista de casos de uso	11
4.1. Diagrama de casos de uso	11
4.2. Descripción de casos de uso	11
4.2.1. CU-01: Crear evento y desplegar contrato	12
4.2.2. CU-02: Compra primaria de ticket	13
4.2.3. CU-03: Listar ticket para reventa	14
4.2.4. CU-04: Compra en reventa atómica	15
4.2.5. CU-05: Verificar ticket en puerta	16
4.2.6. CU-06: Invalidar boleto	17
5. Vista lógica	18
5.1. Arquitectura en capas	18
5.2. Capa de presentación	19
5.2.1. Aplicación Web TuTicket (React + Vite + TypeScript)	19
5.3. Capa de servicios (Backend)	19
5.3.1. Servidor Express (<code>src/server.ts</code>)	19
5.3.2. Indexador Soroban (<code>src/indexer.ts</code>)	20
5.4. Capa de datos (Off-chain)	20
5.5. Capa blockchain (On-chain)	21

5.5.1.	Contrato Factory (FabricaBoletos)	21
5.5.2.	Contrato por Evento (ContratoEvento)	22
5.5.3.	SAC XLM Nativo (Stellar Asset Contract)	25
5.6.	Separación on-chain / off-chain	25
6.	Vista de procesos	26
6.1.	Flujo de compra primaria	26
6.2.	Flujo de reventa atómica	26
6.3.	Flujo de verificación y redención en puerta	28
6.4.	Máquina de estados del ticket	29
6.4.1.	Estados	29
6.4.2.	Transiciones	31
6.4.3.	Reglas de integridad	31
7.	Vista de despliegue	33
7.1.	Configuración de entornos	33
7.2.	Variables de entorno relevantes	34
8.	Vista de implementación	36
8.1.	Organización de repositorios	36
8.2.	Estructura del workspace de contratos	36
8.3.	Estructura del backend	37
8.4.	Estructura del frontend	37
8.5.	Artefactos compilados y comandos de build	37
9.	Vista de datos	38
9.1.	Modelo de datos on-chain: ContratoEvento	38
9.1.1.	Claves de storage (ClaveDato)	38
9.1.2.	Estructura del boleto (Boleto)	38
9.2.	Modelo de datos on-chain: FabricaBoletos	38
9.3.	Modelo de datos off-chain: PostgreSQL	39
9.3.1.	Entidades principales	39
9.3.2.	Correspondencia on-chain / off-chain	40
10.	Atributos de calidad	42
10.1.	QA-01: Seguridad — Autorización por rol	42
10.2.	QA-02: Integridad — Atomicidad en reventa	42
10.3.	QA-03: Trazabilidad — Historial de versiones	42
10.4.	QA-04: Modificabilidad — Separación on-chain/off-chain	43
10.5.	QA-05: Disponibilidad — Resiliencia del backend	43

11.Decisiones arquitectónicas	44
11.1. ADR-01: Selección de Stellar/Soroban como plataforma blockchain	44
11.2. ADR-02: Patrón Factory para contratos de evento	44
11.3. ADR-03: Patrón Burn/Remint para reventa	44
11.4. ADR-04: Separación on-chain/off-chain	45
11.5. ADR-05: Stack tecnológico web	45
11.6. ADR-06: Verificadores autorizados para redención	45

Índice de figuras

1.	Diagrama C4 Nivel 1 – Contexto del sistema Stellar Tickets	3
2.	Diagrama C4 Nivel 2 – Contenedores del sistema Stellar Tickets	5
3.	Diagrama de casos de uso arquitectónicamente significativos	11
4.	Diagrama de secuencia – Reventa atómica con distribución de comisiones .	27
5.	Diagrama de secuencia – Verificación en puerta con escáner QR (Phase 4) .	30
6.	Máquina de estados del ciclo de vida de un ticket	30
7.	Diagrama de despliegue – Topología de nodos en producción (Vercel + Railway/Render + Supabase + Stellar Testnet)	35
8.	Diagrama ERD – Modelo de datos híbrido Web2.5 con campos Web3 . . .	40

Índice de cuadros

3.	Mapeo entre vistas arquitectónicas y secciones del SAD	7
4.	CU-01: Crear evento y desplegar contrato	12
5.	CU-02: Compra primaria de ticket	13
6.	CU-03: Listar ticket para reventa	14
7.	CU-04: Compra en reventa atómica	15
8.	CU-05: Verificar ticket en puerta	16
9.	CU-06: Invalidar boleto	17
10.	Descomposición en capas lógicas	18
11.	Errores tipados del <code>ContratoEvento</code>	24
12.	Distribución de responsabilidades on-chain vs. off-chain	25
13.	Estados del ciclo de vida de un boleto	29
14.	Transiciones válidas entre estados del boleto	31
15.	Nodos de despliegue	33
16.	Variables de entorno principales	34
17.	Repositorios del proyecto	36
18.	Comandos de compilación y artefactos	37
19.	Claves de storage del <code>ContratoEvento</code>	38
20.	Campos del struct <code>Boleto</code>	38
21.	Claves de storage de <code>FabricaBoletos</code>	39
22.	Correspondencia entre datos on-chain y off-chain	41

1. Introducción

1.1. Propósito

Este documento describe la **arquitectura de software** del sistema **Proyecto Tickets-NFT B2B**. Su propósito es:

- Comunicar las decisiones arquitectónicas fundamentales del sistema a todos los interesados.
- Servir como referencia técnica para el equipo de desarrollo durante la implementación.
- Documentar las vistas arquitectónicas que permiten comprender la estructura, comportamiento y despliegue del sistema.
- Justificar las decisiones de diseño y los *trade-offs* realizados.

1.2. Alcance

El SAD abarca la arquitectura del prototipo académico del sistema de reventa B2B de tickets con NFTs sobre Stellar (Soroban), incluyendo:

- El contrato inteligente Soroban (lógica on-chain).
- El backend/API que interactúa con el contrato.
- El frontend web de demostración.
- Las integraciones con la red Stellar (testnet).

1.3. Definiciones, acrónimos y abreviaturas

Término	Definición
SAD	<i>Software Architecture Document</i> . Este documento.
NFT	<i>Non-Fungible Token</i> . Activo digital único que representa un ticket en blockchain.
Stellar	Red blockchain orientada a pagos y emisión de activos digitales.
Soroban	Plataforma de contratos inteligentes sobre Stellar, basada en WebAssembly (Wasm).
WASM	<i>WebAssembly</i> . Formato de bytecode al que se compilan los contratos Soroban.
B2B	<i>Business to Business</i> . Modelo de negocio del sistema.
API	<i>Application Programming Interface</i> .

Término	Definición
REST	<i>Representational State Transfer</i> . Estilo arquitectónico para APIs web.
Testnet	Red de pruebas de Stellar sin valor monetario real.
SRS	<i>Software Requirements Specification</i> . Documento de requerimientos del proyecto.
SPMP	<i>Software Project Management Plan</i> . Plan de gestión del proyecto.

1.4. Visión general del documento

El resto del documento se organiza así:

- **Sección 2:** describe el modelo arquitectónico utilizado y las vistas que se presentan.
- **Sección 3:** enumera los objetivos y restricciones arquitectónicas.
- **Sección 4:** vista de casos de uso.
- **Sección 5:** vista lógica (paquetes, clases, capas).
- **Sección 6:** vista de procesos (conurrencia, flujos).
- **Sección 7:** vista de despliegue (infraestructura física/virtual).
- **Sección 8:** vista de implementación (componentes, módulos).
- **Sección 9:** vista de datos (modelo de datos on-chain y off-chain).
- **Sección 10:** atributos de calidad y tácticas arquitectónicas.
- **Sección 11:** registro de decisiones arquitectónicas (ADRs).

2. Representación arquitectónica

La arquitectura de Stellar Tickets se describe mediante el modelo **C4** de Simon Brown, que provee niveles de abstracción progresivos (contexto, contenedores, componentes, código) para comunicar la arquitectura a audiencias con distintos niveles de detalle técnico. Este modelo es especialmente adecuado para sistemas híbridos como el presente, donde la lógica se distribuye entre contratos inteligentes on-chain y servicios convencionales off-chain.

2.1. Niveles C4 utilizados

Se utilizan los dos primeros niveles del modelo C4:

2.1.1. Nivel 1 – Contexto del sistema

En el modelo C4, el diagrama de contexto es el nivel de abstracción más alto. Su objetivo es responder a la pregunta: *¿qué es el sistema y quién interactúa con él?* No muestra detalles técnicos internos, sino que ubica al sistema como una caja negra dentro de su entorno, identificando las personas que lo usan y los sistemas externos de los que depende o con los que se comunica.

La Figura 1 presenta el diagrama de contexto de Stellar Tickets.

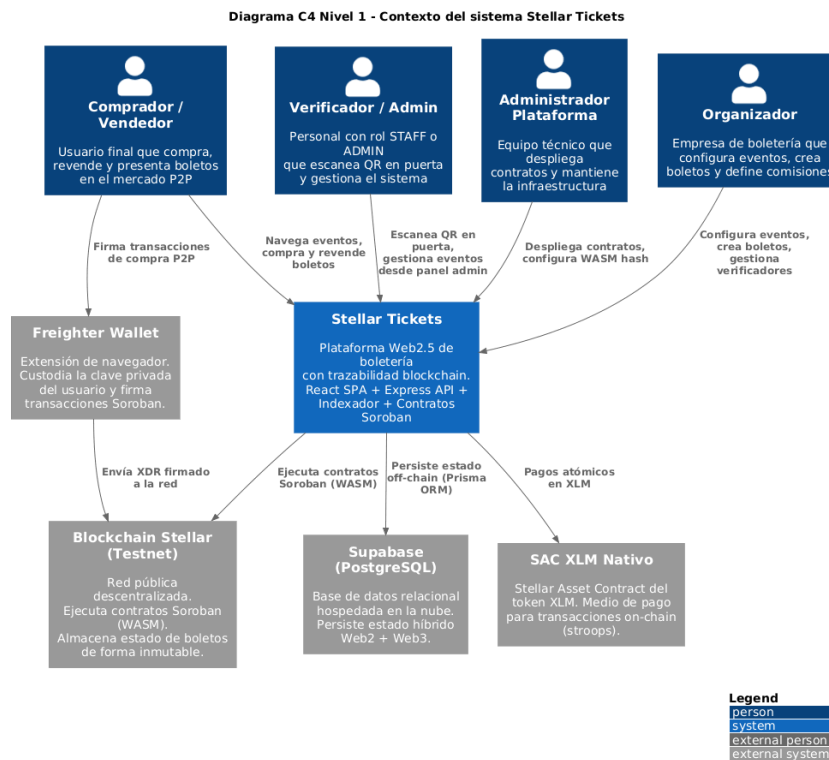


Figura 1: Diagrama C4 Nivel 1 – Contexto del sistema Stellar Tickets

Interpretación del diagrama. El sistema central es la **plataforma Stellar Tickets**, compuesta por un frontend web (React SPA), un backend API (Express) con su Indexador Soroban, y los contratos inteligentes Soroban. Alrededor se identifican los siguientes actores y sistemas externos:

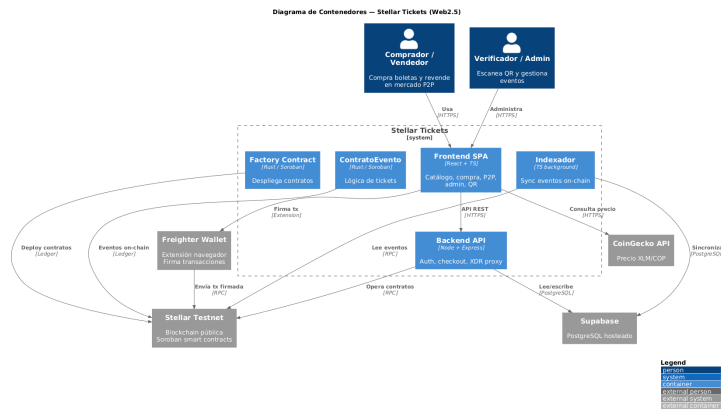
- **Organizador** (empresa de boletería): configura eventos, define comisiones y gestiona boletos a través del panel de administración web. Sus operaciones on-chain son orquestadas por el backend.
- **Comprador** (usuario final): navega eventos, compra boletos en venta primaria o re-venta, y gestiona su inventario de boletos a través de la aplicación web TuTicket.
- **Verificador** (personal del evento): persona autorizada por el organizador para validar boletos en la puerta del evento. Utiliza la funcionalidad de verificación del sistema para marcar boletos como redimidos on-chain.
- **Administrador de la plataforma** (equipo técnico): despliega los contratos en la red Stellar mediante el Factory, configura el WASM hash y mantiene la infraestructura operativa.
- **Blockchain Stellar**: red pública descentralizada que almacena el estado de los contratos de forma inmutable. Los contratos se ejecutan sobre Soroban (motor de smart contracts basado en WASM).
- **SAC XLM Nativo**: Stellar Asset Contract del token XLM nativo de la red Stellar. El contrato de evento lo invoca para ejecutar transferencias atómicas de pago durante compras y reventas.
- **Freighter Wallet**: extensión de navegador que custodia la clave privada del usuario. El frontend la usa para solicitar la firma de transacciones Soroban sin exponer la clave al servidor.

El límite del sistema define qué está bajo control del equipo de desarrollo (frontend, backend, contratos) y qué es externo (red Stellar, SAC XLM, Freighter).

2.1.2. Nivel 2 – Contenedores

El diagrama de contenedores es el segundo nivel del modelo C4. Responde a la pregunta: *¿cuáles son las piezas técnicas que componen el sistema y cómo se comunican entre sí?* Un “contenedor” en C4 no se refiere a Docker, sino a una unidad desplegable independientemente: una aplicación web, una API, una base de datos, un contrato inteligente, etc.

La Figura 2 presenta el diagrama de contenedores de Stellar Tickets.



- **Contrato Factory** (`FabricaBoletos`): contrato Soroban que despliega un contrato de evento independiente por cada evento. Garantiza aislamiento de configuración y fondos entre eventos.
- **Contrato de Evento** (`ContratoEvento`, 11 instancias desplegadas): contrato Soroban desplegado por evento que contiene toda la lógica de negocio on-chain: creación de boletos, venta primaria, reventa atómica con burn/remint, distribución de comisiones y gestión de verificadores. Es el componente más crítico del sistema.
- **SAC XLM Nativo**: Stellar Asset Contract del token XLM nativo. El contrato de evento lo invoca para ejecutar transferencias atómicas de pago.

Comunicaciones clave:

- La Aplicación Web TuTicket se comunica con el Backend API mediante REST/JSON para todas las operaciones (autenticación, navegación, compra, gestión de boletos).
- El Backend API interactúa con PostgreSQL para persistencia off-chain y con el Nodo RPC Soroban para operaciones on-chain.
- El Contrato de Evento invoca al SAC XLM nativo para procesar transferencias de pago de forma atómica dentro de la misma transacción Soroban.
- La Red Stellar subyace como infraestructura que persiste el estado de todos los contratos de forma inmutable y distribuida.

Estado de integración: la conexión Backend API ↔ Soroban está completamente implementada. El backend (Express + TypeScript) ejecuta operaciones on-chain reales sobre la Stellar Testnet: creación de boletos (`crear_boleto`), listado para reventa (`listar_boleto`), cancelación (`cancelar_venta`), construcción de transacciones XDR para que el comprador las firme con Freighter (`comprar_boleto`), y envío de transacciones firmadas. El Indexador sincroniza eventos Soroban hacia PostgreSQL cada 5 segundos.

2.2. Mapeo de vistas a secciones del documento

Las secciones restantes del SAD profundizan en aspectos específicos de la arquitectura, complementando los diagramas C4:

Adicionalmente, el documento incluye una sección de **atributos de calidad** (Sección 10) con escenarios y tácticas arquitectónicas, y un **registro de decisiones arquitectónicas** (Sección 11) en formato ADR.

Vista	Sección	Contenido principal
Casos de uso	4	Casos de uso arquitectónicamente significativos y diagrama C4 de contexto.
Lógica	5	Descomposición en capas y módulos; diagrama C4 de contenedores; separación on-chain/off-chain.
Procesos	6	Flujos de interacción principales: compra primaria, reventa atómica, redención y verificación.
Despliegue	7	Topología de nodos: presentación, servicios, datos y red Stellar testnet.
Implementación	8	Organización del código fuente, artefactos compilados (WASM) y dependencias.
Datos	9	Modelo de almacenamiento on-chain (Soroban) y off-chain (PostgreSQL).

Cuadro 3: Mapeo entre vistas arquitectónicas y secciones del SAD

3. Objetivos y restricciones arquitectónicas

Esta sección presenta los *drivers* arquitectónicos: los atributos de calidad prioritarios y las restricciones que condicionan las decisiones de diseño del sistema.

3.1. Objetivos arquitectónicos

Los siguientes atributos de calidad, derivados del SRS y de las necesidades del modelo B2B, guían las decisiones arquitectónicas:

1. **Trazabilidad e inmutabilidad.** Cada ticket debe tener un historial de propiedad verificable e inalterable. Esto motiva el uso de blockchain como fuente de verdad para la propiedad y las transferencias, complementado con un indexador off-chain para consultas rápidas.
2. **Atomicidad en transacciones de reventa.** La operación de compra en el mercado secundario debe ser atómica: la transferencia del ticket y la distribución de pagos (vendedor, comisión organizador, comisión plataforma) ocurren en una sola transacción o no ocurren en absoluto. Esto elimina estados inconsistentes donde el pago se procesa pero el ticket no se transfiere, o viceversa.
3. **Seguridad y autenticidad.** La arquitectura debe garantizar que los tickets no puedan falsificarse ni duplicarse. La custodia de claves es no-custodial (Freighter Wallet), de modo que solo el propietario legítimo puede autorizar transferencias mediante firma criptográfica.
4. **Adaptabilidad B2B (multi-ticketera).** El sistema debe permitir que múltiples ticketeras operen de forma independiente, cada una con sus propios eventos, políticas de comisión y configuración. Esto motiva la arquitectura de *contrato por evento* desplegado por un Factory. El frontend TuTicket simula una ticketera cliente de la plataforma.
5. **Disponibilidad de consultas.** Aunque la blockchain garantiza persistencia, su modelo de lectura es limitado. El sistema debe ofrecer consultas rápidas sobre estado de tickets, historial y auditoría sin depender exclusivamente de llamadas on-chain. Esto justifica la capa de indexación y caché off-chain respaldada por PostgreSQL.
6. **Verificabilidad en puerta.** La validación de boletos en el punto de acceso al evento se realiza mediante verificadores autorizados por el organizador. El contrato Soroban implementa un registro de verificadores (`Verificador(Address)`) que restringe quién puede marcar un boleto como redimido, garantizando que la redención represente un hecho físico de asistencia.

3.2. Restricciones arquitectónicas

3.2.1. Restricciones tecnológicas

- **Stellar y Soroban:** la lógica on-chain debe implementarse como contratos inteligentes Soroban, compilados a WebAssembly (WASM) y escritos en Rust. Esto impone el uso del `soroban-sdk` y sus limitaciones (modelo de storage basado en claves, sin eventos complejos nativos, tamaño limitado de transacciones).
- **Testnet:** el prototipo opera exclusivamente sobre la testnet de Stellar, lo que implica que los activos no tienen valor monetario real y que la red puede presentar interrupciones o reinicios periódicos.
- **Token de pago:** las transacciones de compra y reventa utilizan **XLM nativo** (Lumens) como token de pago. Los valores se expresan en *stroops* ($1 \text{ XLM} = 10^7 \text{ stroops}$). El Stellar Asset Contract (SAC) nativo (`CDLZFC3S...`) es el contrato de token que invoca el `ContratoEvento`.
- **Firma de transacciones:** operaciones iniciadas por el organizador (`crear_boleto`, `listar_boleto`, `cancelar_venta`) son firmadas server-side con la clave del organizador. La operación de compra (`comprar_boleto`) es firmada por el comprador a través de **Freighter Wallet** (extensión de navegador no-custodial): el backend construye la transacción XDR sin firmar, el frontend la pasa a Freighter para firma, y luego el backend la envía a la red Stellar.

3.2.2. Restricciones organizacionales

- **Equipo reducido:** 4 desarrolladores con dedicación parcial (proyecto académico), lo que favorece una arquitectura con separación clara de responsabilidades que permita trabajo en paralelo (contrato, backend, frontend, verificación).
- **Tiempo acotado:** dos semestres académicos (~ 32 semanas), lo que limita el alcance a un prototipo funcional demostrable.
- **Contexto académico:** el sistema debe ser documentable, demostrable y evaluable por docentes, priorizando claridad arquitectónica sobre optimización de producción.

3.2.3. Restricciones de integración

- **Modelo B2B no invasivo:** el sistema se ofrece como complemento a plataformas de boletería existentes, no como reemplazo. La integración se realiza vía API REST, sin requerir modificaciones en los sistemas internos de la ticketera.

- **Separación on-chain/off-chain:** las operaciones de escritura que afectan propiedad y pagos ocurren on-chain (fuente de verdad), mientras que las consultas de lectura, auditoría e indexación se resuelven off-chain para rendimiento y usabilidad.

4. Vista de casos de uso

Esta sección identifica los casos de uso arquitectónicamente significativos del sistema: aquellos que ejercitan los principales componentes, imponen restricciones de diseño o exponen los atributos de calidad prioritarios descritos en la Sección 3.

4.1. Diagrama de casos de uso

La Figura 3 presenta los casos de uso principales del sistema y sus relaciones con los actores identificados en el diagrama C4 de contexto (Figura 1).

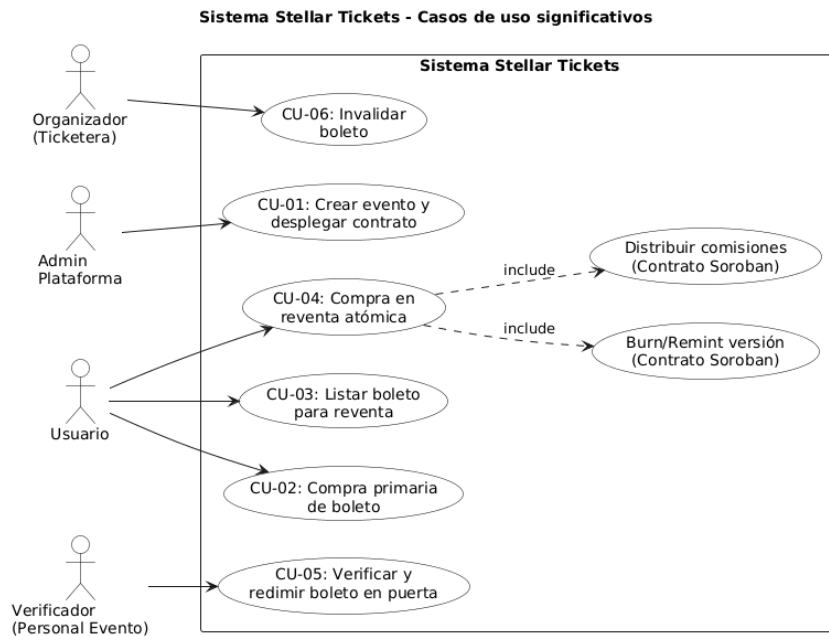


Figura 3: Diagrama de casos de uso arquitectónicamente significativos

4.2. Descripción de casos de uso

A continuación se describe cada caso de uso, destacando su relevancia arquitectónica.

4.2.1. CU-01: Crear evento y desplegar contrato

Actores	Admin Plataforma
Descripción	El administrador de la plataforma configura un nuevo evento (nombre, fecha, wallets de comisión, políticas de reventa). La plataforma invoca el Factory para desplegar un contrato Soroban específico para ese evento en la testnet.
Precondiciones	El administrador está autenticado y tiene permisos de creación de eventos.
Flujo principal	1. El administrador ingresa los datos del evento y configura fees. 2. El backend valida los parámetros y registra el evento en PostgreSQL. 3. El backend invoca el contrato Factory en Stellar. 4. El Factory despliega un nuevo contrato de evento con los parámetros configurados. 5. El sistema registra la dirección del contrato desplegado.
Postcondiciones	Existe un contrato de evento independiente en la testnet, listo para emitir tickets.
Relevancia arquitectónica	Ejerce el patrón Factory, la separación on-chain/off-chain y la adaptabilidad multi-ticketera. Cada evento opera con un contrato aislado, lo que garantiza independencia de configuración y fondos.

Cuadro 4: CU-01: Crear evento y desplegar contrato

4.2.2. CU-02: Compra primaria de ticket

Actores	Usuario
Descripción	El usuario adquiere un ticket emitido por el organizador. El 100 % del pago se transfiere al organizador.
Precondiciones	El evento existe, tiene tickets disponibles y el usuario tiene fondos suficientes en su wallet.
Flujo principal	1. El usuario selecciona un boleto disponible en la aplicación web TuTicket. 2. El frontend envía la solicitud de compra al Backend API. 3. El backend orquesta la invocación de <code>comprar_boleto</code> en el contrato de evento. 4. El contrato transfiere el 100 % del pago al organizador mediante <code>CienteToken</code> y asigna el boleto al usuario, marcando <code>es_reventa = true</code>. 5. El backend registra la operación en PostgreSQL.
Postcondiciones	El usuario es el nuevo propietario del boleto. El pago fue transferido al organizador. Futuras ventas de este boleto serán tratadas como reventa (con comisiones y burn/remint).
Relevancia arquitectónica	Ejerce la orquestación Backend→Soroban, la escritura on-chain con registro off-chain posterior, y la transición <code>es_reventa = false</code> → <code>true</code> que determina el comportamiento de todas las transacciones futuras sobre ese boleto.

Cuadro 5: CU-02: Compra primaria de ticket

4.2.3. CU-03: Listar ticket para reventa

Actores	Usuario
Descripción	El propietario de un ticket lo publica en el marketplace para reventa, estableciendo un precio.
Precondiciones	El usuario es propietario del ticket, el ticket está vigente (no consumido ni invalidado) y el evento permite reventa.
Flujo principal	1. El usuario selecciona un boleto de su inventario y fija precio de reventa. 2. El frontend envía la solicitud al Backend API. 3. El backend invoca <code>listar_boleto</code> en el contrato de evento, que marca <code>en_venta = true</code> y actualiza el precio. 4. El marketplace muestra el boleto disponible para compra.
Postcondiciones	El boleto está visible en el marketplace con estado <code>en_venta = true</code> .
Relevancia arquitectónica	Ejerce la transición de estados del ticket on-chain y la sincronización con el caché off-chain para consultas del marketplace.

Cuadro 6: CU-03: Listar ticket para reventa

4.2.4. CU-04: Compra en reventa atómica

Actores	Usuario
Descripción	El usuario adquiere un ticket en el mercado secundario. En una sola transacción atómica se ejecutan: pago al vendedor, comisión al organizador, comisión a la plataforma, burn de la versión anterior y mint de una nueva versión del ticket.
Precondiciones	El ticket está listado para reventa, el usuario tiene fondos suficientes y no es el propietario actual.
Flujo principal	1. El usuario selecciona un boleto en reventa y confirma la compra. 2. El frontend envía la solicitud al Backend API. 3. El backend invoca <code>comprar_boleto</code> en el contrato de evento, que ejecuta atómicamente: a) Transferencia de comisión al organizador (<code>ComisionOrganizador%</code>). b) Transferencia de comisión a la plataforma (<code>ComisionPlataforma%</code>). c) Transferencia del pago neto al vendedor. d) Burn: la versión actual se marca como <code>invalidado = true</code>. e) Remint: se crea nueva versión (<code>version + 1</code>) asignada al usuario. 4. El contrato emite evento <code>BoletoRevendido</code> on-chain indexado por <code>ticket_root_id</code>. 5. El backend actualiza PostgreSQL.
Flujo alternativo	Si cualquier paso falla, toda la transacción se revierte. No existe estado intermedio.
Postcondiciones	El usuario posee la nueva versión del ticket. Los pagos y comisiones fueron distribuidos. La versión anterior está invalidada.
Relevancia arquitectónica	Es el caso de uso más crítico del sistema. Ejerce la atomicidad transaccional de Soroban, el patrón burn+remint, la distribución de comisiones embebida en el contrato y la trazabilidad por <code>ticket_root_id</code> . Valida el driver de atomicidad e inmutabilidad.

Cuadro 7: CU-04: Compra en reventa atómica

4.2.5. CU-05: Verificar ticket en puerta

Actores	Personal del Evento
Descripción	Un verificador autorizado por el organizador valida el boleto del asistente en la puerta del evento y lo marca como redimido de forma irreversible en el contrato.
Precondiciones	El boleto existe, no está invalidado, no ha sido redimido previamente, y el verificador está registrado como autorizado en el contrato (<code>es_verificador = true</code>).
Flujo principal	1. El asistente presenta su boleto (código QR) en la puerta del evento. El QR contiene un payload JSON con el <code>ticketId</code> (UUID en PostgreSQL). 2. El verificador (rol <code>STAFF</code> o <code>ADMIN</code>) abre la página <code>/escanear</code> en su dispositivo. 3. La cámara del dispositivo lee el QR. El frontend envía <code>POST /api/admin/scan {ticketId}</code> al Backend API. 4. El backend valida que el boleto existe en PostgreSQL con estado <code>ACTIVE</code>. 5. Si es válido, actualiza el estado a <code>CANCELLED</code> en PostgreSQL y retorna <code>{success: true}</code>. 6. La interfaz muestra “Acceso Permitido” (verde) al verificador.
Postcondiciones	El boleto queda marcado como <code>CANCELLED</code> en PostgreSQL. No puede volver a escanearse.
Relevancia arquitectónica	Ejerce el sistema de roles off-chain (<code>STAFF/ADMIN</code>), la validación de estado en PostgreSQL y la decisión de diseño de scan DB-only. El contrato Soroban implementa <code>redimir_boleto</code> y el sistema de verificadores on-chain como mecanismo complementario de trazabilidad criptográfica, activable en producción donde la latencia adicional sea aceptable.

Cuadro 8: CU-05: Verificar ticket en puerta

4.2.6. CU-06: Invalidar boleto

Actores	Organizador
Descripción	El organizador invalida un boleto de forma administrativa (evento cancelado, error de emisión, etc.), marcándolo como <code>invalidado = true</code> on-chain.
Precondiciones	El boleto existe y no ha sido invalidado previamente. El actor es el organizador del evento.
Flujo principal	1. El organizador solicita la invalidación del boleto desde el panel de administración. 2. El backend invoca <code>invalidar_boleto</code> en el contrato de evento. 3. El contrato marca el boleto como <code>invalidado = true</code>, <code>en_venta = false</code> (irreversible). 4. El contrato emite evento <code>BoletoInvalidadoEvt</code>. 5. El backend actualiza el estado en PostgreSQL.
Postcondiciones	El boleto no puede ser listado, comprado, revendido ni redimido. El estado es terminal.
Relevancia arquitectónica	Ejerce la invalidación on-chain (a diferencia de versiones anteriores donde se manejaba solo off-chain), la irreversibilidad de estado y la autorización exclusiva del organizador para operaciones administrativas.

Cuadro 9: CU-06: Invalidar boleto

5. Vista lógica

Esta sección describe la descomposición lógica del sistema en capas y módulos, profundizando el diagrama C4 de contenedores (Figura 2). La separación fundamental es entre la lógica **on-chain** (ejecutada de forma descentralizada en la red Stellar) y la lógica **off-chain** (ejecutada en infraestructura controlada por la plataforma).

5.1. Arquitectura en capas

El sistema se organiza en cuatro capas lógicas, cada una con responsabilidades claramente delimitadas:

1. **Capa de presentación** (Frontend): interfaz de usuario que permite a los distintos actores interactuar con el sistema.
2. **Capa de servicios** (Backend/API): lógica de negocio off-chain, orquestación de operaciones, autenticación y exposición de endpoints.
3. **Capa de datos** (PostgreSQL + Caché): persistencia off-chain para consultas rápidas, auditoría e indexación.
4. **Capa blockchain** (Stellar/Soroban): lógica de negocio on-chain, fuente de verdad para propiedad de tickets y pagos.

Capa	Contenedores	Responsabilidad
Presentación	Aplicación Web Tu-Ticket (React/Vite)	Renderizar interfaces, capturar acciones del usuario, comunicarse con el Backend API.
Servicios	Backend API (Express + Indexador Soroban)	Autenticar usuarios (JWT/bcrypt), orquestar operaciones de negocio, gestionar carrito, procesar checkout, exponer API REST. Firma server-side de <code>crear_boleto/listar_boleto/cancelar_venta</code> . Construcción de XDR para firma por Freightier. Sincronización de eventos on-chain mediante indexador en segundo plano.
Datos	PostgreSQL (schemas: public, ticketing)	Persistir usuarios, eventos, sedes, boletos, órdenes, pagos y auditoría. Fuente de datos para consultas del frontend.
Blockchain	Contrato Factory (FabricaBoletos), Contrato por Evento (ContratoEvento), SAC XLM nativo	Crear boletos, transferir propiedad, distribuir comisiones, redimir y invalidar boletos. Fuente de verdad inmutable. 11 contratos desplegados en Stellar Testnet.

Cuadro 10: Descomposición en capas lógicas

5.2. Capa de presentación

La capa de presentación consiste en una aplicación web:

5.2.1. Aplicación Web TuTicket (React + Vite + TypeScript)

Aplicación SPA (*Single Page Application*) que simula una ticketera cliente de la plataforma. Utiliza React 18 con Vite como bundler, TypeScript, Tailwind CSS para estilos y shadcn/ui (basado en Radix UI) como librería de componentes. Expone las siguientes funcionalidades:

- **Comprador:** navegación de eventos por categoría, búsqueda, selección de boletos y asientos, carrito de compras, checkout en 3 pasos (datos del comprador → método de pago → confirmación), historial de compras y gestión de boletos propios.
- **Administrador:** panel de administración para gestión de eventos, creación de tipos de boletos, consulta de estadísticas de ventas y ocupación.

La comunicación con la blockchain **no es directa**: el frontend se comunica con el Backend API mediante llamadas REST/JSON autenticadas con JWT para la mayoría de operaciones. La excepción es la **firma de transacciones de compra en reventa**: el frontend recibe el XDR sin firmar del backend y lo pasa a la extensión **Freighter Wallet** del navegador para obtener la firma del comprador, sin que el backend tenga acceso a claves privadas del usuario.

Rutas principales (23 rutas): / (inicio), /eventos (catálogo), /eventos/:categoria, /buscar, /evento/:id (detalle + marketplace P2P), /evento/:id/boletas, /evento/:id/asientos, /carrito, /checkout, /confirmacion, /login, /registro, /mi-cuenta (perfil, entradas, compras, **ventas P2P**), /admin (panel de administración, solo rol ADMIN), /escanear (escáner QR de verificación, roles ADMIN y STAFF).

5.3. Capa de servicios (Backend)

El backend es una aplicación **Express 4 + TypeScript** que centraliza la lógica de negocio off-chain en un único proceso. No utiliza NestJS; la arquitectura es un servidor Express monolítico (`src/server.ts`) con un proceso de indexación en segundo plano (`src/indexer.ts`).

5.3.1. Servidor Express (`src/server.ts`)

Implementa todos los endpoints REST en un único archivo. Se organiza conceptualmente en los siguientes grupos de rutas:

- **Auth:** POST /api/auth/login, POST /api/auth/register. Contraseñas con hash bcrypt (10 rounds), tokens JWT (7 días de expiración). Si NODE_ENV=production y no se configura JWT_SECRET, el servidor rechaza arrancar.
- **Usuarios:** GET /api/users/me, PATCH /api/users/me, PATCH /api/users/me/wallet. Roles: CUSTOMER, ADMIN, STAFF.
- **Eventos:** listado con caché en memoria (60 s), detalle por slug (30 s), destacados, tipos de boleto, eventos relacionados.
- **Carrito:** CRUD de ítems con verificación de propiedad por usuario (un usuario no puede modificar el carrito de otro).
- **Checkout:** previsualización y confirmación atómica (crea orden, ítems, boletos y pago en una sola transacción Prisma). Pago simulado como PAID (prototipo académico).
- **Órdenes y boletos:** historial de compras, boletos activos con campos Web3, boletos vendidos en P2P.
- **Transacciones Soroban:** secure-ticket (organizer-signed crear_boleto), list-ticket (listar_boleto), cancel-listing (cancelar_venta), build-buy-xdr (construye XDR sin firmar para Freightier), submit (envía XDR firmado a la red).
- **Admin (ADMIN):** gestión de eventos, venues, contratos desplegados y escaneo QR. Protegido por verificación del rol en cada endpoint (re-consulta la BD).
- **Admin scan (ADMIN/STAFF):** POST /api/admin/scan recibe un ticketId, valida que el boleto exista y esté activo, y lo marca como CANCELLED en PostgreSQL. Nota de diseño: el scan es **DB-only** (sin llamada on-chain a redimir_boleto) para minimizar la latencia en puerta; se trata de un trade-off documentado.

5.3.2. Indexador Soroban (src/indexer.ts)

Proceso de fondo que sondea el RPC de Soroban cada 5 segundos. Procesa 7 tipos de eventos on-chain y actualiza PostgreSQL para mantener consistencia. Persiste el cursor en la tabla `indexer_state` y se recupera automáticamente si el cursor cae fuera del rango de ledgers retenidos por el RPC público. No se ejecuta en entornos serverless (VERCEL=1).

5.4. Capa de datos (Off-chain)

La capa de datos off-chain se compone de una base de datos PostgreSQL 16 alojada en **Supabase**, gestionada mediante Prisma ORM. Utiliza un único schema `ticketing`:

- **users:** id, email (único), password_hash (bcrypt), first_name, last_name, phone, document_type (CC/CE/TI/PP), document_number, role (CUSTOMER | ADMIN | STAFF), wallet_address (dirección Freightier, única, nullable).

- **events:** id, title, slug, status (DRAFT/PUBLISHED/CANCELLED/SOLD_OUT), starts_at, ends_at, organizer_id, venue_id, category_id, contract_address (dirección del ContratoEvento desplegado en Testnet, nullable).
- **tickets:** id, order_item_id (FK), status (ACTIVE/USED/CANCELLED/REFUNDED), contract_address, ticket_root_id (u32, ID estable on-chain), version (u32, se incrementa en cada reventa), is_for_sale (bool), resale_price (BigInt, stroops), owner_wallet (dirección del propietario en Stellar). Restricción única: (contract_address, ticket_root_id, version).
- **indexer_state:** cursor de sincronización del indexador (last_ledger).
- **Comercio Web2:** organizers, venues, venue_sections, seats, event_ticket_types, event_seat_inventory (AVAILABLE/HELD/SOLD/BLOCKED), carts, cart_items, orders, order_items, payments (provider_reference almacena el hash de transacción Soroban), event_categories, cities.

PostgreSQL es la fuente de datos para consultas del frontend. Los datos on-chain se sincronizan en tiempo real mediante el Indexador Soroban que actualiza los campos Web3 de la tabla tickets.

5.5. Capa blockchain (On-chain)

La capa on-chain es la fuente de verdad del sistema para la propiedad de boletos y la ejecución de pagos. Todo el código de los contratos está escrito en español (nombres de structs, funciones, variables y errores). Se compone de:

5.5.1. Contrato Factory (FabricaBoletos)

Contrato Soroban responsable de desplegar un contrato independiente por cada evento. Implementa el patrón Factory para garantizar aislamiento de configuración, storage y fondos entre eventos.

Modelo de datos. Claves de storage (ClaveDato): Administrador, ContadorEventos, HashWasmEvento (hash del WASM compilado del ContratoEvento), ContratoEvento(u32) (mapeo id_evento → dirección), ContratoRegistrado(Address) (mapeo inverso para evitar duplicados).

Configuración de evento. El struct ConfiguracionEvento empaqueta: id_evento, organizador, token_pago, comision_organizador (u32, 0–99), comision_plataforma (u32, 0–99), wallet_organizador, wallet_plataforma, capacidad_total.

Funciones públicas:

- `inicializar(administrador)`: setup único de la factory. Requiere auth del administrador.
- `configurar_wasm_evento(hash)`: almacena el hash WASM del `ContratoEvento`. Requiere auth del administrador.
- `crear_evento_contrato(config, dir_prueba) → Address`: despliega e inicializa un nuevo contrato de evento. Requiere **dobles autorizaciones**: administrador y organizador. Valida comisiones $< 100\%$, capacidad > 0 , `id_evento` no duplicado.
- `obtener_contrato_evento(id_evento) → Address`: consulta la dirección del contrato de un evento.
- `obtener_contador_eventos() → u32`: total de eventos creados.

Deploy programático. En producción, usa `deployer().with_current_contract(salt).deploy_v2(hash, ())` para crear contratos en la blockchain con dirección determinada derivada del `id_evento`. En tests, usa una dirección pre-registrada (Soroban testutils no soporta deploy real).

5.5.2. Contrato por Evento (ContratoEvento)

Contrato Soroban que implementa toda la lógica de negocio on-chain para un evento específico. Su estructura interna se organiza de la siguiente forma:

Modelo de datos (Storage). El contrato utiliza un almacenamiento basado en claves (`ClaveDato`) con los siguientes elementos:

- `Boleto(u32, u32)`: almacena el struct `Boleto` por `(ticket_root_id, version)`. Cada boleto contiene: `ticket_root_id` (ID estable que no cambia entre reventas), `version` (se incrementa con cada reventa: 0, 1, 2...), `id_evento`, `propietario` (Address), `precio` (i128), `en_venta`, `es_reventa`, `usado`, `invalidado`.
- `VersionActual(u32)`: puntero a la versión vigente de un boleto. Se actualiza en cada reventa.
- `ContadorBoletos`: contador autoincremental que asigna el próximo `ticket_root_id`.
- `Organizador, Plataforma`: direcciones de la ticketera y de la plataforma.
- `TokenPago`: dirección del SAC XLM nativo (CDLZFC3SYJYDZT7K67VZ75HPJVIEUVNIXF47ZG2FB2RMQQVU2HHGCYSC).
- `ComisionOrganizador, ComisionPlataforma`: porcentajes de comisión (i128, base 100).

- `Verificador(Address)`: registra si una dirección tiene permiso para redimir boletos en puerta.

Funciones de escritura (state-changing). Cada función está protegida por `require_auth()` que verifica la firma del actor autorizado:

- `inicializar`: configura el contrato con las direcciones, token de pago y porcentajes de comisión. Solo puede ejecutarse una vez (guarda de re-inicialización). Requiere auth del organizador.
- `crear_boleto(id_evento, precio) → u32`: emite un nuevo boleto (version 0) asignado al organizador. Requiere auth del organizador. Emite evento `BoletoCreado`.
- `listar_boleto(ticket_root_id, nuevo_precio)`: marca un boleto como en venta y actualiza su precio. Requiere auth del propietario. Valida que no esté ya en venta, ni usado, ni invalidado. Emite evento `BoletoListado`.
- `cancelar_venta(ticket_root_id)`: retira un boleto del marketplace. Requiere auth del propietario. Emite evento `VentaCancelada`.
- `comprar_boleto(ticket_root_id, comprador) → u32`: ejecuta la compra. Implementa dos flujos en una sola transacción atómica:
 - *Venta primaria* (`es_reventa = false`): el 100 % del pago se transfiere al organizador mediante `CienteToken`. El boleto se actualiza in-place: nuevo propietario, `es_reventa = true`. Retorna la versión actual. Emite evento `BoletoCompradoPrimario`.
 - *Reventa* (`es_reventa = true`): el pago se distribuye en tres transferencias atómicas: comisión al organizador, comisión a la plataforma y remanente al vendedor. Luego se ejecuta **burn/remint**: la versión actual se marca como `invalidado = true`, se crea una nueva versión (`version + 1`) asignada al comprador, y se actualiza el puntero `VersionActual`. Retorna la nueva versión. Emite evento `BoletoRevendido`.

Requiere auth del comprador. Valida que el comprador no sea el propietario actual (`AutoCompra`).

- `agregar_verificador(address)`: registra un verificador autorizado. Requiere auth del organizador. Emite evento `VerificadorAgregado`.
- `remove_verificador(address)`: desautoriza un verificador. Requiere auth del organizador. Emite evento `VerificadorRemovido`.
- `redimir_boleto(ticket_root_id, verificador)`: marca el boleto como `usado = true` (irreversible). **Solo verificadores autorizados** pueden invocarla (no el propietario). Requiere auth del verificador. Emite evento `BoletoRedimido`.

- `invalidar_boleto(ticket_root_id)`: cancela un boleto administrativamente (`invalidado = true`). Requiere auth del organizador. Emite evento `BoletoInvalidadoEvt`.

Funciones de lectura (views). No requieren autorización:

- `obtener_boleto(ticket_root_id)`: retorna el struct `Boleto` de la versión vigente.
- `obtener_boleto_version(ticket_root_id, version)`: retorna una versión específica (historial).
- `obtener_version_vigente(ticket_root_id)`: retorna el número de versión actual.
- `obtener_propietario(ticket_root_id)`: retorna la dirección del propietario actual.
- `obtener_boletos_reventa()`: retorna todos los boletos en reventa activa ($O(n)$, usar indexador off-chain en producción).
- `obtener_boletos_evento(id_evento)`: retorna todos los boletos de un evento ($O(n)$).
- `es_verificador(address)`: consulta si una dirección es verificador autorizado.

Errores tipados (ErrorContrato). El contrato define 16 errores tipados como enum con `#[contracterror]` y `#[repr(u32)]`, lo que permite manejo programático off-chain sin parsear strings:

Código	Nombre	Significado
1	<code>YaInicializado</code>	Doble inicialización del contrato
2	<code>ComisionesMuyAltas</code>	Suma de comisiones $\geq 100\%$
3	<code>ComisionesNegativas</code>	Comisión menor a 0
4	<code>PrecioInvalido</code>	Precio ≤ 0
5	<code>YaEnVenta</code>	Boleto ya listado
6	<code>BoletoUsado</code>	Boleto ya redimido
7	<code>NoEnVenta</code>	Boleto no está en venta
8	<code>AutoCompra</code>	Comprador = propietario
9	<code>YaUsado</code>	Segundo intento de redención
10	<code>BoletoNoEncontrado</code>	ID o versión inexistente
11	<code>NoAutorizado</code>	Sin permisos para la acción
12	<code>VersionInvalida</code>	Versión no existe
13	<code>BoletoInvalidado</code>	Boleto cancelado/burned
14	<code>NoInicializado</code>	Contrato sin inicializar
15	<code>VerificadorYaExiste</code>	Dirección ya es verificador
16	<code>VerificadorNoEncontrado</code>	Dirección no es verificador

Cuadro 11: Errores tipados del `ContratoEvento`

Eventos on-chain. El contrato emite eventos tipados con `#[contractevent]` para indexación off-chain. Cada evento tiene hasta 4 topics indexados:

BoletoCreado, BoletoListado, VentaCancelada, BoletoCompradoPrimario, BoletoRevendido, BoletoRedimido, BoletoInvalidadoEvt, VerificadorAgregado, VerificadorRemovido.

5.5.3. SAC XLM Nativo (Stellar Asset Contract)

Contrato externo que implementa el estándar de token de Stellar para el XLM nativo. El contrato de evento lo invoca mediante `ClienteToken` (alias de `soroban_sdk::token::Client`) para ejecutar las transferencias de pago. Se usa el SAC del XLM nativo, cuya dirección en testnet es `CDLZFC3S...HHGCYSC`. Los precios se expresan en *stroops* (1 XLM = 10^7 stroops).

5.6. Separación on-chain / off-chain

La siguiente tabla resume el criterio de distribución de responsabilidades entre ambas capas:

Responsabilidad	On-chain	Off-chain
Propiedad de boletos (quién es dueño)	✓	Copia indexada
Ejecución de pagos y comisiones	✓	—
Creación de boletos (mint)	✓	Registro en PostgreSQL
Redención (marcar como usado)	✓	Copia indexada
Invalidación administrativa	✓	Copia indexada
Gestión de verificadores autorizados	✓	—
Historial de versiones (burn/remint)	✓	Consulta indexada
Despliegue de contratos por evento	✓ (Factory)	—
Consultas de búsqueda y filtrado	—	✓
Autenticación y roles de usuario	—	✓
Carrito y seat holds	—	✓
Gestión de eventos, sedes y asientos	—	✓

Cuadro 12: Distribución de responsabilidades on-chain vs. off-chain

El principio rector es: *lo que afecta propiedad o dinero se ejecuta on-chain* (fuente de verdad inmutable), *lo que requiere consultas rápidas o contexto de negocio se resuelve off-chain* (rendimiento y usabilidad).

6. Vista de procesos

Esta sección describe los flujos de interacción principales del sistema, enfocándose en cómo los componentes colaboran en tiempo de ejecución para resolver los casos de uso críticos. Se documentan tres flujos y la máquina de estados que gobierna el ciclo de vida del ticket.

6.1. Flujo de compra primaria

La compra primaria es el primer flujo que ocurre sobre un boleto recién creado. Involucra al comprador, el frontend TuTicket, el Backend API, el contrato de evento y el token de pago.

1. El organizador ha creado previamente el evento y emitido boletos mediante `crear_boleto`. Cada boleto queda asignado al organizador con `en_venta = false` y `es_reventa = false`.
2. El organizador lista el boleto para venta primaria invocando `listar_boleto`, que establece `en_venta = true` y el precio.
3. El comprador visualiza el boleto en la aplicación web TuTicket y selecciona la compra.
4. El frontend envía la solicitud al Backend API, que orquesta la invocación de `comprar_boleto` en el contrato de evento.
5. El contrato ejecuta la lógica de compra primaria: como `es_reventa = false`, el 100 % del pago se transfiere al organizador mediante `ClienteToken`.
6. El contrato actualiza el boleto: `propietario = comprador`, `en_venta = false`, `es_reventa = true` (todas las ventas futuras serán reventas). Emite evento `BoletoCompradoPrimario`.
7. El backend registra la operación en PostgreSQL.

Aspecto arquitectónico clave: la compra primaria establece la transición de `es_reventa = false` a `true`, lo que determina el comportamiento de todas las transacciones futuras sobre ese boleto (activación del split de comisiones y del mecanismo burn/remint).

6.2. Flujo de reventa atómica

La Figura 4 presenta el diagrama de secuencia completo de este flujo.

Este es el flujo más crítico del sistema desde el punto de vista arquitectónico. La reventa debe ser **atómica**: todas las operaciones (distribución de pagos, burn de la versión anterior y remint de nueva versión) ocurren en una sola transacción de Soroban, o ninguna ocurre. No existe estado intermedio.

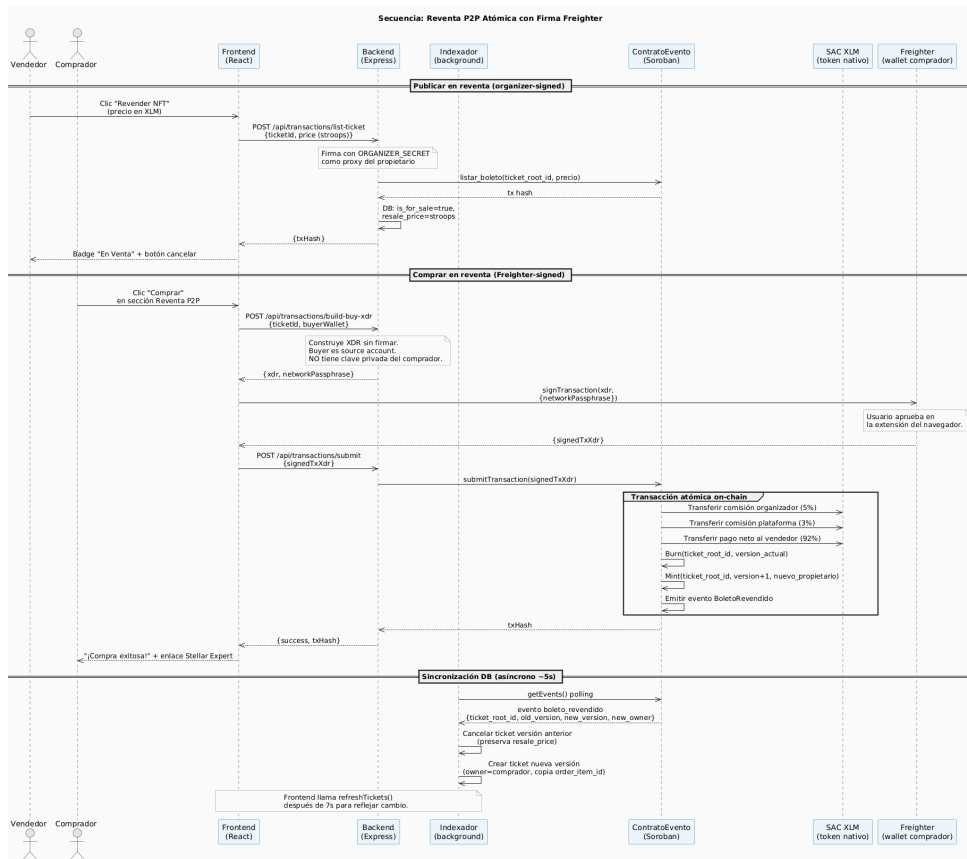


Figura 4: Diagrama de secuencia – Reventa atómica con distribución de comisiones

1. **Publicación:** el vendedor (propietario actual) invoca `listar_boleto` con el nuevo precio. El contrato verifica que el vendedor es el propietario (`require_auth`), que el boleto no está ya en venta, ni usado, ni invalidado. El boleto queda con `en_venta = true`. El contrato emite evento `BoletoListado`.
2. **Compra:** el comprador selecciona el boleto en el marketplace e inicia la compra. El frontend envía la solicitud al Backend API, que orquesta la invocación de `comprar_boleto`.
3. **Ejecución atómica en el contrato:** como `es_reventa = true`, el contrato ejecuta la distribución de comisiones y el burn/remint en una sola transacción:
 - a) Calcula la comisión del organizador: $\text{precio} * \text{ComisionOrganizador} / 100$.
 - b) Calcula la comisión de la plataforma: $\text{precio} * \text{ComisionPlataforma} / 100$.
 - c) Calcula el pago neto al vendedor: $\text{precio} - \text{comisión_org} - \text{comisión_plat}$.
 - d) Ejecuta tres transferencias mediante `ClienteToken`: al organizador, a la plataforma y al vendedor.
 - e) **Burn:** marca la versión actual del boleto como `invalidado = true`, `en_venta = false`.

- f) **Remint:** crea nueva versión (`version + 1`) con `propietario = comprador`, `en_venta = false`, `es_reventa = true`.
 - g) Actualiza el puntero `VersionActual` a la nueva versión.
4. **Evento on-chain:** el contrato emite `BoletoRevendido` con `ticket_root_id`, `vendedor`, `comprador`, `precio`, `version_anterior` y `version_nueva`.
 5. **Registro off-chain:** el backend registra la transacción en PostgreSQL.
 6. **Notificación:** el frontend muestra al comprador su nuevo boleto y notifica al vendedor que la venta se completó.

Garantía de atomicidad: si cualquiera de las transferencias de pago falla (por ejemplo, fondos insuficientes del comprador), toda la transacción se revierte automáticamente por el runtime de Soroban. Esto elimina por diseño los estados inconsistentes donde el pago se procesa pero el boleto no se transfiere, o viceversa. No se requiere lógica de compensación ni rollback manual.

Trazabilidad por versionado: la combinación (`ticket_root_id`, `version`) permite reconstruir el historial completo de propiedad de un boleto consultando todas sus versiones. Las versiones anteriores quedan como registros históricos inmutables con `invalidado = true`.

6.3. Flujo de verificación y redención en puerta

La verificación de acceso al evento involucra a un verificador autorizado que valida el boleto del asistente y lo marca como redimido on-chain.

1. El organizador ha creado previamente una cuenta con rol **STAFF** (o **ADMIN**) en el sistema, asignada al personal de verificación.
2. El asistente presenta su boleto (código QR) en la puerta del evento. El QR codifica un payload JSON con el `ticketId` (UUID del boleto en PostgreSQL).
3. El verificador utiliza la página `/escanear` del sistema (accesible desde el navegador del dispositivo de verificación) con su sesión activa.
4. La cámara del dispositivo lee el QR. El frontend envía `POST /api/admin/scan {ticketId}` al Backend API.
5. El backend valida: el boleto existe en PostgreSQL, tiene estado **ACTIVE**. Si no, retorna error con mensaje descriptivo.
6. Si es válido, el backend actualiza el estado a **CANCELLED** en PostgreSQL y retorna `{success: true}`.

7. La interfaz del escáner muestra “Acceso Permitido” (verde) o “Acceso Denegado” (rojo) con el código del boleto.

Nota de implementación: el escáner actual opera en modo **DB-only** (sin llamada on-chain a `redimir_boleto`). Esta decisión de diseño minimiza la latencia en puerta (evita el round-trip al RPC Soroban de ~ 5 segundos). El contrato Soroban implementa la función `redimir_boleto` y el sistema de verificadores autorizados on-chain como mecanismo de validación criptográfica, que puede activarse en una versión de producción donde la latencia adicional sea aceptable.

Aspecto arquitectónico clave: el sistema de verificadores on-chain (registro en el contrato Soroban) y la gestión de roles off-chain (STAFF en PostgreSQL) son mecanismos complementarios que refuerzan la separación entre propietario (custodia del ticket) y personal autorizado (capacidad de redención).

La Figura 5 presenta el diagrama de secuencia completo de este flujo.

6.4. Máquina de estados del ticket

El ciclo de vida de un ticket está gobernado por una máquina de estados que define las transiciones válidas entre estados y las condiciones bajo las cuales pueden ocurrir. Esta máquina se implementa implícitamente en el contrato Soroban mediante las validaciones en cada función. La Figura 6 presenta el diagrama de estados completo.

6.4.1. Estados

Estado	Descripción
Emitido	El boleto acaba de ser creado por el organizador (<code>crear_boleto</code>). Está asignado al organizador, con <code>en_venta = false</code> , <code>es_reventa = false</code> , <code>usado = false</code> , <code>invalidado = false</code> .
En venta	El boleto ha sido listado para venta o reventa (<code>listar_boleto</code>). <code>en_venta = true</code> . Visible en el marketplace.
Asignado	El boleto tiene un propietario (comprador) y no está en venta. <code>en_venta = false</code> . El propietario puede listarlo para reventa o presentarlo en la puerta del evento.
Consumido	El boleto fue redimido por un verificador autorizado (<code>redimir_boleto</code>). <code>usado = true</code> . Estado terminal e irreversible. No puede listarse, comprarse ni revenderse.
Invalidado	El boleto fue cancelado administrativamente por el organizador (<code>invalidar_boleto</code>) o fue “quemado” (<i>burned</i>) durante una reventa para crear la nueva versión. <code>invalidado = true</code> . Estado terminal on-chain.

Cuadro 13: Estados del ciclo de vida de un boleto

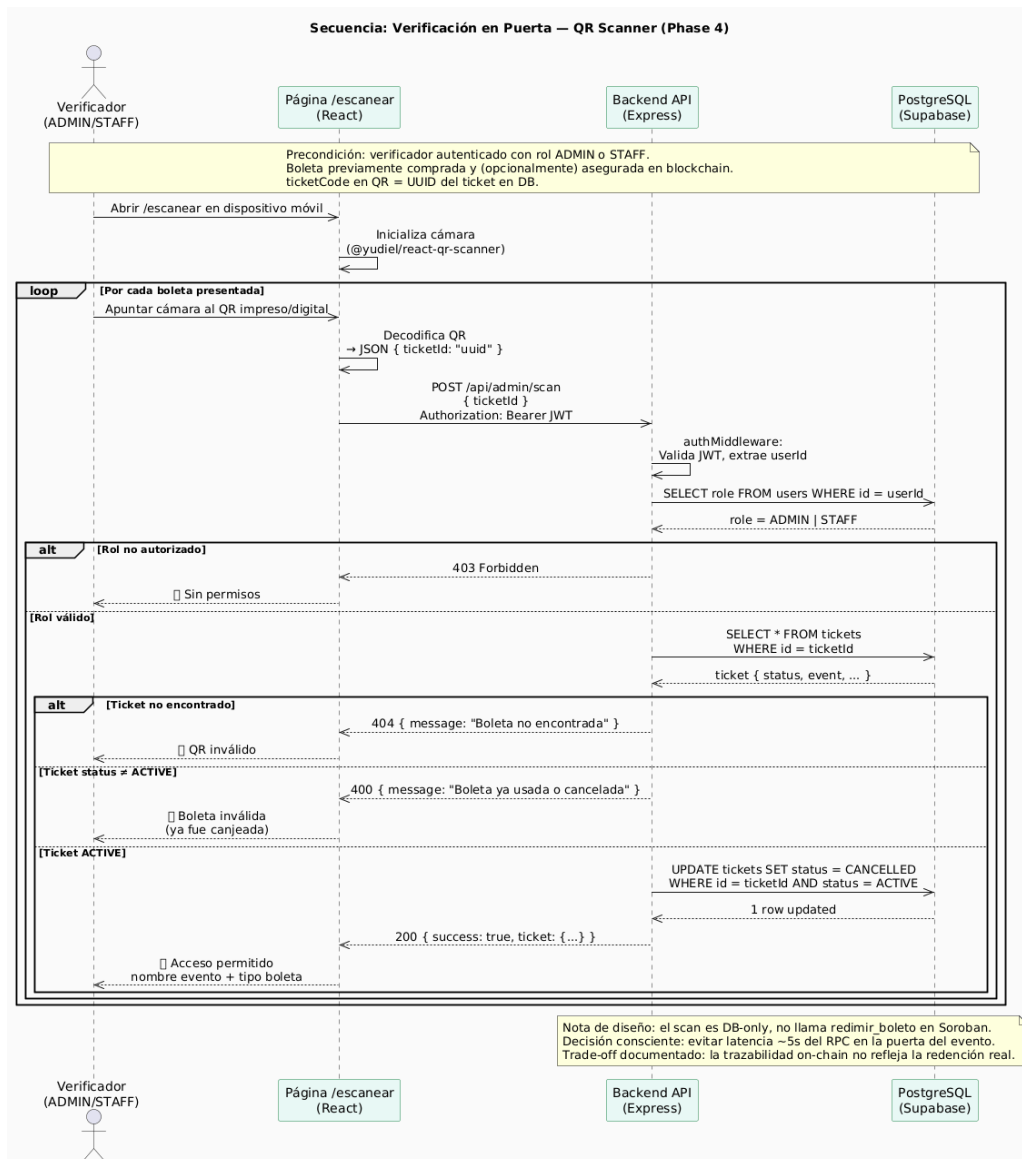


Figura 5: Diagrama de secuencia – Verificación en puerta con escáner QR (Phase 4)

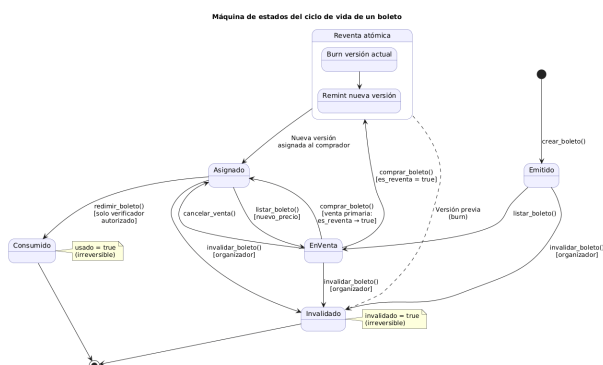


Figura 6: Máquina de estados del ciclo de vida de un ticket

6.4.2. Transiciones

Desde	Hacia	Acción	Condiciones
Emitido	En venta	<code>listar_boleto</code>	El organizador (propietario) lista el boleto para venta primaria.
En venta	Asignado	<code>comprar_boleto</code>	Venta primaria: 100 % al organizador, <code>es_reventa</code> pasa a <code>true</code> . Update in-place.
Asignado	En venta	<code>listar_boleto</code>	El propietario lista el boleto para reventa. No debe estar usado ni invalidado.
En venta	Asignado	<code>cancelar_venta</code>	El propietario retira el boleto del marketplace.
En venta	Asignado (nueva versión)	<code>comprar_boleto</code>	Reventa: distribución de comisiones + burn versión actual + remint nueva versión.
Asignado	Consumido	<code>redimir_boleto</code>	Un verificador autorizado redime el boleto. Irreversible (<code>usado = true</code>).
No invalidado	Invalidado	<code>invalidar_boleto</code>	El organizador cancela el boleto on-chain. Irreversible (<code>invalidado = true</code>).
En venta (re-venta)	Invalidado	<code>comprar_boleto</code>	La versión previa se invalida (burn) como parte del flujo de reventa.

Cuadro 14: Transiciones válidas entre estados del boleto

6.4.3. Reglas de integridad

Las siguientes reglas son impuestas por el contrato y no pueden ser violadas:

- Un boleto con `usado = true` no puede cambiar a ningún otro estado. La redención es **irreversible**.
- Un boleto con `invalidado = true` no puede venderse, comprarse, listarse ni redimirse. La invalidación es **irreversible**.
- Solo el propietario actual puede invocar `listar_boleto` y `cancelar_venta` (verificado por `require_auth`).
- Solo un **verificador autorizado** (registrado on-chain) puede invocar `redimir_boleto`. El propietario no puede auto-redimirse.
- Un propietario no puede comprarse su propio boleto (`AutoCompra` está prohibido).
- El contrato solo puede inicializarse una vez (error `YaInicializado`).

- Las comisiones de organizador y plataforma no pueden sumar 100 % o más (**ComisionesMuyAltas**).
- Cada versión de un boleto es inmutable una vez invalidada: las versiones anteriores quedan como registro histórico en el storage.

7. Vista de despliegue

El sistema se despliega en cuatro servicios cloud independientes que conforman la topología de producción.

Cuadro 15: Nodos de despliegue

Nodo	Plataforma	Componentes
Cliente	Navegador (Chrome + Freighter)	SPA React servida como archivos estáticos. Extensión Freighter gestiona clave privada del usuario.
Frontend	Vercel (CDN estático)	Bundle /frontend/dist. SPA con 23 rutas client-side.
Backend + Indexador	Railway / Render (proceso long-lived)	<code>tsx src/server.ts</code> : API Express + Indexador Soroban en background. El Indexador requiere proceso persistente (no compatible con serverless).
Base de datos	Supabase (PostgreSQL 16)	Schema <code>ticketing</code> : 26+ tablas Web2+Web3. Pool de conexiones vía <code>pgbouncer</code> (<code>connection_limit=5</code>).
Blockchain	Stellar Testnet (Soroban)	<code>FabricaBoletos</code> + 11 instancias de <code>ContratoEvento</code> + SAC XLM nativo.

7.1. Configuración de entornos

El sistema distingue dos modos de ejecución para el backend:

- **Long-lived (Railway/Render)**: ejecuta `tsx src/server.ts`. Arranca `app.listen` (PORT) e inicia el Indexador Soroban en segundo plano.
- **Serverless (Vercel)**: la variable `VERCEL=1` desactiva `app.listen()` y el Indexador. El backend se exporta como handler via `backend/api/index.ts`.

En desarrollo local, el frontend corre con `npm run dev` (Vite, puerto 8080) y el backend con `npm run dev` (tsx watch, puerto 3000).

7.2. Variables de entorno relevantes

Cuadro 16: Variables de entorno principales

Variable	Servicio	Descripción
DATABASE_URL	Backend	Cadena de conexión PostgreSQL con Supabase (Prisma, <code>connection_limit=5</code>).
JWT_SECRET	Backend	Secreto para firma de tokens JWT. Obligatorio en producción; falla al arrancar si no está definido.
SOROBAN_RPC_URL	Backend	URL del nodo RPC de Soroban (https://soroban-testnet.stellar.org).
ORGANIZER_SECRET	Backend	Clave privada del organizador para firmar server-side <code>crear_boleto</code> , <code>listar_boleto</code> y <code>cancelar_venta</code> .
FACTORY_CONTRACT_ID	Backend	ID del contrato <code>FabricaBoletos</code> desplegado. Requerido por el panel de administración.
VITE_API_BASE_URL	Frontend	URL base del backend API (por defecto <code>http://localhost:3000</code>).

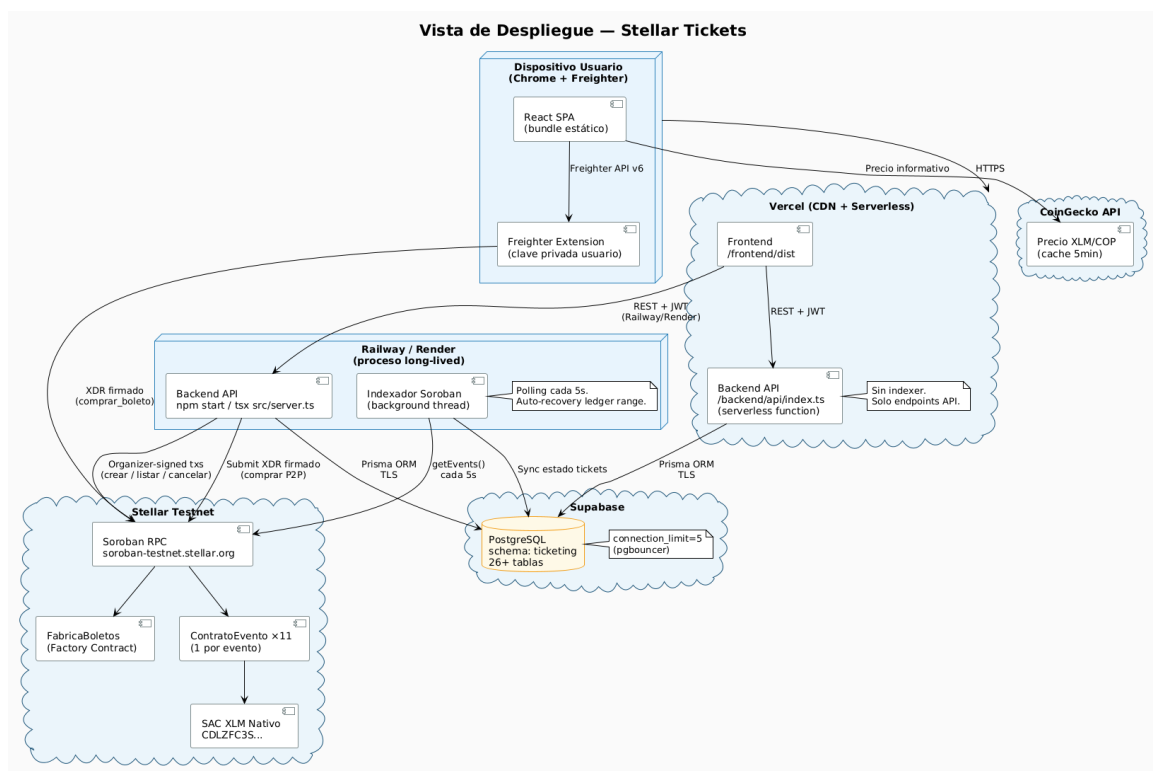


Figura 7: Diagrama de despliegue – Topología de nodos en producción (Vercel + Railway/Render + Supabase + Stellar Testnet)

8. Vista de implementación

8.1. Organización de repositorios

El proyecto se organiza en un **monorepo** principal que contiene contratos, backend y frontend, más un repositorio separado para la documentación:

Cuadro 17: Repositorios del proyecto

Repositorio	Ruta local	Contenido
main_contract	codigo/main_contract/	Monorepo: contratos Soroban (Rust), backend Express (Node.js) y frontend React.
Documento_Tesis	Documentos/Documento_Tesis/	Documentación LaTeX del proyecto.

La estructura interna del monorepo `main_contract` es:

```
main_contract/
+-- contracts/      # Cargo workspace: event_contract + factory_contract
+-- backend/        # Node.js Express + Prisma + Indexador Soroban
+-- frontend/       # React 18 + Vite + TypeScript + Freightier
+-- docs/           # Documentación técnica (architecture, ADRs, backlog)
+-- .github/        # CI: build contratos + tests cargo
```

8.2. Estructura del workspace de contratos

El repositorio de contratos utiliza un workspace de Cargo con dos crates:

```
contracts/
+-- Cargo.toml      # Workspace raiz
+-- contracts/
  +-- event_contract/
    | +-- src/
    |   +-- lib.rs    # ContratoEvento (~1140 lineas)
    |   +-- test.rs   # Suite de tests
  +-- factory_contract/
    +-- src/
      +-- lib.rs      # FabricaBoletos (~430 lineas)
      +-- test.rs     # Suite de tests
```

Dependencia: `factory_contract` depende de `event_contract` (importa `ContratoEventoClient` para inicializar contratos hijos).

8.3. Estructura del backend

```
src/
+-- app.module.ts      # Modulo raiz
+-- auth/              # Autenticacion JWT (registro, login, guards)
+-- users/             # Gestion de usuarios y perfiles
+-- events/            # CRUD de eventos
+-- tickets/           # Gestion de boletos y disponibilidad
+-- cart/              # Carrito de compras (sesion)
+-- checkout/          # Procesamiento de compras (simulado)
+-- admin/             # Panel de administracion
+-- prisma/            # Servicio Prisma y migraciones
+-- common/            # Guards, decoradores, DTOs compartidos
```

8.4. Estructura del frontend

```
src/
+-- main.tsx           # Entry point
+-- App.tsx            # Router principal
+-- components/        # Componentes reutilizables (shadcn/ui)
+-- pages/             # Vistas por ruta
+-- services/          # Llamadas HTTP al backend (Axios)
+-- context/           # React Context (auth, carrito)
+-- hooks/             # Custom hooks
+-- types/             # Interfaces TypeScript
```

8.5. Artefactos compilados y comandos de build

Cuadro 18: Comandos de compilación y artefactos

Componente	Comando	Artefacto
Contratos	<code>stellar contract build</code>	<code>target/wasm32v1-none/release/{event,factory}_contract.wasm</code>
Backend	<code>npm run build</code>	<code>dist/</code> (JavaScript transpilado)
Frontend	<code>npm run build</code>	<code>dist/</code> (SPA estática optimizada)
Tests contratos	<code>cargo test</code>	Resultados de pruebas unitarias Soroban (48+ tests)

9. Vista de datos

El sistema mantiene dos modelos de datos complementarios: on-chain (inmutable, en los contratos Soroban) y off-chain (relacional, en PostgreSQL).

9.1. Modelo de datos on-chain: ContratoEvento

9.1.1. Claves de storage (ClaveDato)

Cuadro 19: Claves de storage del ContratoEvento

Clave	Tipo de valor	Descripción
Organizador	Address	Dirección del organizador del evento.
Plataforma	Address	Wallet de la plataforma para comisiones.
TokenPago	Address	Contrato del token usado para pagos.
ComisionOrganizador	i128	Porcentaje de comisión del organizador.
ComisionPlataforma	i128	Porcentaje de comisión de la plataforma.
ContadorBoletos	u32	ID incremental para nuevos boletos.
Boleto(u32)	Boleto	Datos de un boleto por su ID compuesto.
VersionActual(u32)	u32	Versión vigente de un <code>ticket_root_id</code> .
EsVerificador(Address)	bool	Indica si una dirección es verificador autorizado.

9.1.2. Estructura del boleto (Boleto)

Cuadro 20: Campos del struct Boleto

Campo	Tipo	Descripción
<code>ticket_root_id</code>	u32	Identificador raíz (invariante entre versiones).
<code>version</code>	u32	Número de versión (incrementa con cada reventa).
<code>id_evento</code>	u32	ID del evento al que pertenece.
<code>propietario</code>	Address	Dirección del dueño actual.
<code>precio</code>	i128	Precio en unidades del token de pago.
<code>en_venta</code>	bool	Si está listado para reventa.
<code>es_reventa</code>	bool	<code>true</code> después de la primera compra.
<code>usado</code>	bool	Si fue redimido (irreversible).
<code>invalidado</code>	bool	Si fue invalidado por el organizador (irreversible).

9.2. Modelo de datos on-chain: FabricaBoletos

El contrato `FabricaBoletos` mantiene un registro centralizado de todos los eventos desplegados y su configuración. Su storage es de sólo escritura para el administrador y de lectura pública para el backend.

Cuadro 21: Claves de storage de FabricaBoletos

Clave	Tipo de valor	Descripción
Administrador	Address	Admin de la factory.
ContadorEventos	u32	Total de eventos creados.
HashWasmEvento	BytesN<32>	Hash del WASM del ContratoEvento.
ContratoEvento(u32)	Address	Maapeo id_evento → dirección del contrato.
ContratoRegistrado(Address)	bool	Maapeo inverso para evitar duplicados.

9.3. Modelo de datos off-chain: PostgreSQL

La base de datos relacional utiliza el schema `ticketing` en PostgreSQL 16 (hosteado en Supabase):

9.3.1. Entidades principales

El esquema contiene las siguientes tablas, con sus campos Web3 añadidos como capa de extensión sobre el modelo relacional clásico, con sus campos Web3 añadidos como capa de extensión sobre el modelo relacional clásico:

- **users**: id, email (único), password_hash (bcrypt), nombre, teléfono, tipo y número de documento, rol (CUSTOMER | ADMIN | STAFF), wallet_address (dirección pública Freighter, único, opcional).
- **events**: id, título, slug (único), descripción, fecha, estado (DRAFT | PUBLISHED | CANCELLED), venue, organizador, categoría, contract_address (dirección del ContratoEvento en Soroban, único, opcional).
- **tickets**: id, estado (ACTIVE | USED | CANCELLED | REFUNDED), order_item_id (FK), contract_address, ticket_root_id (u32), version (u32), is_for_sale (bool), resale_price (BIGINT en stroops), owner_wallet. Restricción de unicidad: (contract_address, ticket_root_id, version).
- **orders / order_items**: historial de compras fiat. subtotal, service_fees, total.
- **payments**: método (CARD | PSE | CASHPOINT), estado, monto, provider_reference (hash de tx Soroban cuando aplica).
- **carts / cart_items**: carrito de compras por usuario.
- **event_ticket_types**: tipos de boleta por evento (nombre, precio, cantidad).
- **indexer_state**: cursor del Indexador Soroban (last_ledger), un único registro.

La Figura 8 presenta el diagrama entidad-relación de las tablas clave con sus campos Web3.

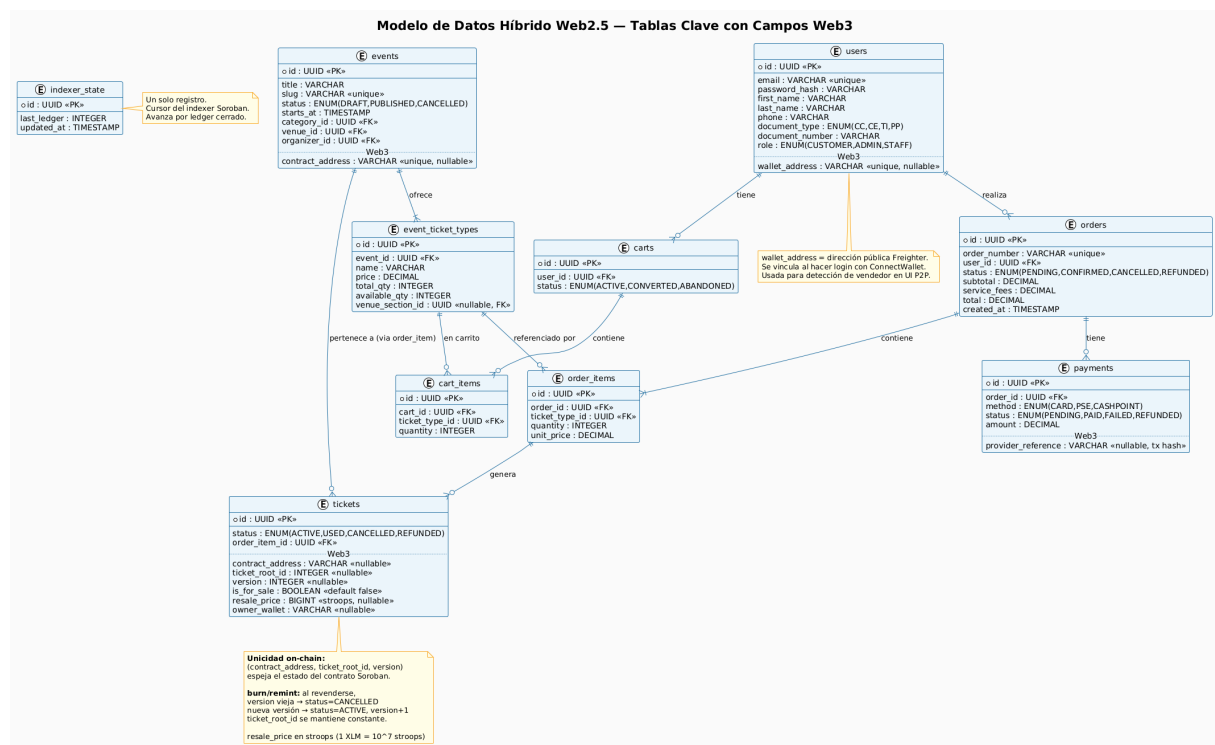


Figura 8: Diagrama ERD – Modelo de datos híbrido Web2.5 con campos Web3

9.3.2. Correspondencia on-chain / off-chain

Nota: La integración Soroban está completamente implementada. Los campos `contract_address`, `ticket_root_id`, `version`, `is_for_sale`, `resale_price` y `owner_wallet` en la tabla `tickets` reflejan el estado on-chain, sincronizado por el Indexador Soroban. La columna `wallet_address` en `users` vincula la identidad Web2 con la dirección Stellar del usuario.

Cuadro 22: Correspondencia entre datos on-chain y off-chain

Concepto	On-chain	Off-chain
Evento	ContratoEvento (contrato por evento)	Tabla <code>events</code> + campo <code>contract_address</code>
Boleto	Boleto struct: <code>ticket_root_id</code> , <code>version</code> , propietario	Tabla <code>tickets</code> + campos <code>ticket_root_id</code> , <code>version</code> , <code>owner_wallet</code>
Propietario	propietario (Address Stellar)	<code>owner_wallet</code> en <code>tickets</code> + <code>wallet_address</code> en <code>users</code>
En venta	<code>en_venta</code> (bool)	<code>is_for_sale</code> (bool) + <code>resale_price</code> (BIGINT, stroops)
Estado de uso	<code>usado</code> / <code>invalidado</code> (bool)	<code>status</code> : <code>ACTIVE</code> / <code>USED</code> / <code>CANCELLED</code>
Compra fiat	No aplica	Tablas <code>orders</code> , <code>order_items</code> , <code>payments</code>
Compra on-chain	Transferencia atómica XLM + burn/remint	Sincronizado por Indexador vía evento <code>boleto_revendido</code>
Comisiones	Split automático en <code>comprar_boleto</code>	No almacenado (derivable desde precio y porcentajes del contrato)

10. Atributos de calidad

Los siguientes escenarios de calidad guían las decisiones arquitectónicas del sistema.

10.1. QA-01: Seguridad — Autorización por rol

Aspecto	Descripción
Estímulo	Un usuario intenta ejecutar una operación para la cual no tiene permisos.
Fuente	Actor externo (usuario, contrato, llamada API).
Artefacto	Contratos Soroban + Backend Express.
Respuesta	La operación es rechazada sin modificar el estado.
Medida	100 % de las funciones que mutan estado validan autorización.

Táctica: On-chain, cada función pública usa `require_auth()` sobre la dirección correspondiente (organizador, propietario, verificador, admin). Off-chain, Express usa `authMiddleware` (valida Bearer JWT y adjunta `userId`) y los endpoints admin reconsultan el rol desde PostgreSQL en cada solicitud.

10.2. QA-02: Integridad — Atomicidad en reventa

Aspecto	Descripción
Estímulo	Un comprador invoca <code>comprar_boleto</code> sobre un boleto en reventa.
Artefacto	<code>ContratoEvento</code> : función <code>comprar_boleto</code> .
Respuesta	Se transfieren tokens (vendedor + comisiones) y se crea nueva versión del boleto en una sola transacción atómica. Si cualquier paso falla, toda la transacción se revierte.
Medida	Cero estados intermedios observables: o se completa todo, o no se modifica nada.

Táctica: Soroban garantiza atomicidad transaccional nativa. El patrón burn/remint invalida la versión anterior y crea la nueva en la misma invocación, eliminando la posibilidad de doble gasto.

10.3. QA-03: Trazabilidad — Historial de versiones

Táctica: El `ticket_root_id` se mantiene constante a través de todas las versiones. Las versiones anteriores permanecen en el storage como registros históricos inmutables. Los eventos on-chain proporcionan un log de auditoría adicional.

Aspecto	Descripción
Estímulo	Un auditor desea rastrear la cadena de propiedad de un boleto.
Artefacto	Storage on-chain del <code>ContratoEvento</code> .
Respuesta	Se puede reconstruir el historial completo consultando todas las versiones desde (<code>ticket_root_id</code> , 0) hasta <code>VersionActual</code> .
Medida	Cada transferencia de propiedad genera una versión inmutable con evento on-chain (<code>BoletoRevendido</code>).

Aspecto	Descripción
Estímulo	Se requiere cambiar la interfaz de usuario o agregar un nuevo módulo al backend.
Artefacto	Frontend React, Backend Express.
Respuesta	El cambio se realiza sin modificar los contratos inteligentes.
Medida	Los contratos definen reglas de negocio invariantes; la lógica de presentación y orquestación vive off-chain y es modificable independientemente.

10.4. QA-04: Modificabilidad — Separación on-chain/off-chain

Táctica: Arquitectura en capas con separación estricta. El backend actúa como intermediario y puede evolucionar sin requerir un redploy de los contratos. El patrón Factory permite desplegar nuevas versiones del `ContratoEvento` sin afectar los contratos existentes.

10.5. QA-05: Disponibilidad — Resiliencia del backend

Aspecto	Descripción
Estímulo	El servidor backend se reinicia o pierde conexión temporalmente.
Artefacto	Backend Express + PostgreSQL (Supabase).
Respuesta	Los datos transaccionales persisten en PostgreSQL. Los contratos on-chain siguen operativos independientemente. Al recuperarse el backend, retoma operación normal.
Medida	Pérdida de datos cero para transacciones confirmadas en PostgreSQL. Los contratos on-chain tienen disponibilidad inherente de la red Stellar.

Táctica: PostgreSQL provee durabilidad ACID. La blockchain Stellar tiene alta disponibilidad intrínseca (consenso federado, sin downtime planificado). El backend es stateless (sesiones en JWT) y puede reiniciarse sin pérdida de contexto.

11. Decisiones arquitectónicas

11.1. ADR-01: Selección de Stellar/Soroban como plataforma blockchain

Contexto: Se necesita una plataforma blockchain que soporte contratos inteligentes con costos de transacción bajos, finalidad rápida y un ecosistema maduro de tokens.

Decisión: Se selecciona Stellar con Soroban (contratos inteligentes en Rust compilados a WASM) por: transacciones con finalidad en 5 segundos, costos de gas del orden de fracciones de centavo, soporte nativo de tokens (Stellar Asset Contract), y atomicidad transaccional garantizada.

Consecuencias: El equipo debe aprender Rust y el SDK de Soroban. El ecosistema de herramientas es más joven que el de Ethereum/Solidity, pero suficiente para el alcance del proyecto. Las pruebas se ejecutan con el entorno simulado de Soroban SDK.

11.2. ADR-02: Patrón Factory para contratos de evento

Contexto: El sistema debe soportar múltiples eventos, cada uno con su propia configuración de comisiones, token de pago y organizador.

Decisión: Se implementa un contrato `FabricaBoletos` que despliega instancias independientes de `ContratoEvento` mediante `deploy_v2`. Cada evento tiene su propio contrato con storage aislado.

Consecuencias: Aislamiento total entre eventos (un bug en un contrato no afecta a otros). El storage de Soroban tiene límites por contrato, y este patrón evita cuellos de botella. La factory mantiene un registro de todos los contratos desplegados para descubrimiento. El costo es un deploy adicional por evento.

11.3. ADR-03: Patrón Burn/Remint para reventa

Contexto: Al revender un boleto, se necesita transferir propiedad manteniendo trazabilidad completa del historial.

Decisión: En lugar de mutar el propietario del boleto existente, se invalida la versión actual (`invalidado = true`) y se crea una nueva versión con `version + 1`, nuevo propietario y nuevo precio. El `ticket_root_id` se mantiene constante.

Consecuencias: Trazabilidad completa: cada versión anterior permanece como registro histórico inmutable en el storage. Se puede reconstruir toda la cadena de propiedad. El costo es mayor uso de storage (una entrada por versión), pero el beneficio en auditoría y prevención de fraude justifica el trade-off.

11.4. ADR-04: Separación on-chain/off-chain

Contexto: No toda la lógica del sistema necesita vivir en la blockchain. El almacenamiento on-chain es costoso y las operaciones son lentas comparadas con una base de datos tradicional.

Decisión: Se mantiene on-chain únicamente la lógica crítica de negocio (propiedad, transferencias, comisiones, redención). La gestión de usuarios, catálogo de eventos, carrito de compras, y lógica de presentación vive off-chain en Express + PostgreSQL.

Consecuencias: Mejor experiencia de usuario (operaciones off-chain son instantáneas). Menor costo de gas. La consistencia entre ambos mundos se mantiene mediante el Indexador Soroban, que hace polling cada 5 segundos y sincroniza 7 tipos de eventos on-chain hacia PostgreSQL con auto-recovery de rango de ledgers.

11.5. ADR-05: Stack tecnológico web

Contexto: Se necesita un frontend y backend web para la aplicación cliente TuTicket.

Decisión: Frontend con React 18 + Vite + TypeScript + Tailwind CSS + shadcn/ui + @stellar/freighter-api. Backend con Express 4 + TypeScript + Prisma ORM + PostgreSQL (Supabase) + @stellar/stellar-sdk. Autenticación con JWT de token único (7 días de expiración, bcrypt para contraseñas).

Consecuencias: TypeScript compartido entre frontend y backend reduce el cambio de contexto. Prisma genera tipos automáticamente a partir del schema. shadcn/ui acelera el desarrollo de UI con componentes accesibles. Express permite un servidor monolítico simple con el Indexador Soroban corriendo en el mismo proceso, adecuado para la escala del prototipo de tesis.

11.6. ADR-06: Verificadores autorizados para redención

Contexto: Se necesita un mecanismo para redimir boletos en la puerta del evento que sea seguro y no dependa de que el propietario auto-redima su boleto (lo cual podría ser manipulado).

Decisión: El organizador registra direcciones de verificadores autorizados on-chain (agregar_verificador). Solo estos verificadores pueden invocar redimir_boleto. El propietario del boleto no puede auto-redimirse.

Consecuencias: Separación de responsabilidades clara: el propietario posee el boleto, pero no controla su redención. Previene fraudes donde un propietario podría intentar redimir y luego reclamar que no lo hizo. Los verificadores son gestionables (agregar/remove) por el organizador.

Referencias

- [1] IEEE Std 1471-2000: *IEEE Recommended Practice for Architectural Description of Software-Intensive Systems*.
- [2] ISO/IEC/IEEE 42010:2011: *Systems and software engineering — Architecture description*.
- [3] Brown, S. (2018). *The C4 Model for Visualising Software Architecture*. <https://c4model.com/>.
- [4] Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice*, 3rd Edition. Addison-Wesley.
- [5] SRS del sistema Tickets-NFT B2B (v0.1).
- [6] SPMP del sistema Tickets-NFT B2B (v1.1).
- [7] Stellar Development Foundation: *Stellar Documentation*, <https://developers.stellar.org/>.
- [8] Soroban Documentation: *Smart Contracts on Stellar*, <https://soroban.stellar.org/>.